

Requirement:

Due to the increasing demand for online food delivery services, a large food delivery platform has decided to automate its order processing system. The platform receives orders from multiple restaurants simultaneously, and processing all orders sequentially has become too slow. As a result, the platform decided to hire N delivery dispatchers such that each dispatcher is responsible only for processing a single batch of orders. Each dispatcher is required to calculate the total price of each order in their batch by summing the prices of all items in the order. If the total price of an order is **greater than or equal** to the minimum order value V , then the order is eligible for free delivery. Otherwise, a delivery fee is charged. Finally, each dispatcher should report the number of orders eligible for free delivery from their assigned batch. The platform manager should receive the count of eligible orders from each dispatcher.

Write a C program that simulates this scenario using **forking concepts** with a single parent process (delivery platform) and N children processes (dispatchers).

The program accepts **three** arguments from the command line:

1. A file path indicating the orders of all customers on the platform. All prices are non-negative integer values.
2. N : The number of dispatchers (**$N > 0$**)
3. V : The minimum order value for free delivery. (**$V > 0$**)

Input File Format:

- The first line in the file indicates the number of orders (M) in the list.
- Each line in the following M lines represents a single order. It consists of two integers separated by a space. The first integer is the number of items in the order and the second integer is the price per item. The total price of an order is calculated by multiplying these two values.

Output Format:

- The program should output **ONE** line **ONLY** as follows:

The output line consists of N integers separated by a space such that each integer represents the number of free-delivery-eligible orders in the dispatcher's assigned batch. The first dispatcher is assigned the first $\lfloor M/N \rfloor$ orders, the second dispatcher is assigned the second $\lfloor M/N \rfloor$ orders, and so on. **The output should be ordered correctly.** The number of dispatchers is not necessarily divisible by the number of orders. In this case, the overflowing orders are assigned to the N -th dispatcher (the last one).

Example:

```
gcc orderProcessor.c -o orderProcessor.o  
./orderProcessor.o "orders.txt" 3 50
```

Example for “orders.txt”:

```
9
3 10
5 20
2 30
4 5
6 8
1 70
3 15
2 40
7 6
```

Expected output:

```
2 1 1
```

Submission:

You are required to submit **ONE FILE** called **firstName_secondName_ID.c**

It will be evaluated as follows:

```
gcc firstName_secondName_ID.c -o firstName_secondName_ID.o
```

```
./firstName_secondName_ID.o "something.txt" N V -> This will be done over several test cases
```

Notes:

1. The output of your program **should be written exactly** as shown in the expected output.
2. Do necessary printing for debugging your code (or better use a debugger). Do not forget to comment these printf functions before submission.
3. Do not resort to any user inputs from the standard input (i.e. Do not use scanf/gets/etc.). Your code will report a time-out exception in the test cases. The inputs should be given as arguments from the terminal.
4. All the test cases are valid inputs. You do not need to check for any invalid inputs.
5. You are required to use forking in this lab. You have to decide the parent's responsibilities and each child's responsibilities. You also have to find out how you will communicate between both processes.
6. **IMPORTANT NOTE (1):** Do not assume any assumptions about the process IDs of the children. For example, do not assume that the first forked process will have PID+1 where PID is the process ID of the parent process and so on.
7. **IMPORTANT NOTE (2):** Keep in mind that if a parent process creates a process, then the PID of the child process is not necessarily greater than that of the parent. Sometimes, the system wraps around to reusing the free lower PIDs. Therefore, do not make any assumptions about the sequence of children PIDs. Do not say the PID of the first forked child is surely less than that of the second forked one.
8. For forking, you may use **ONLY** the functions specified in this document.